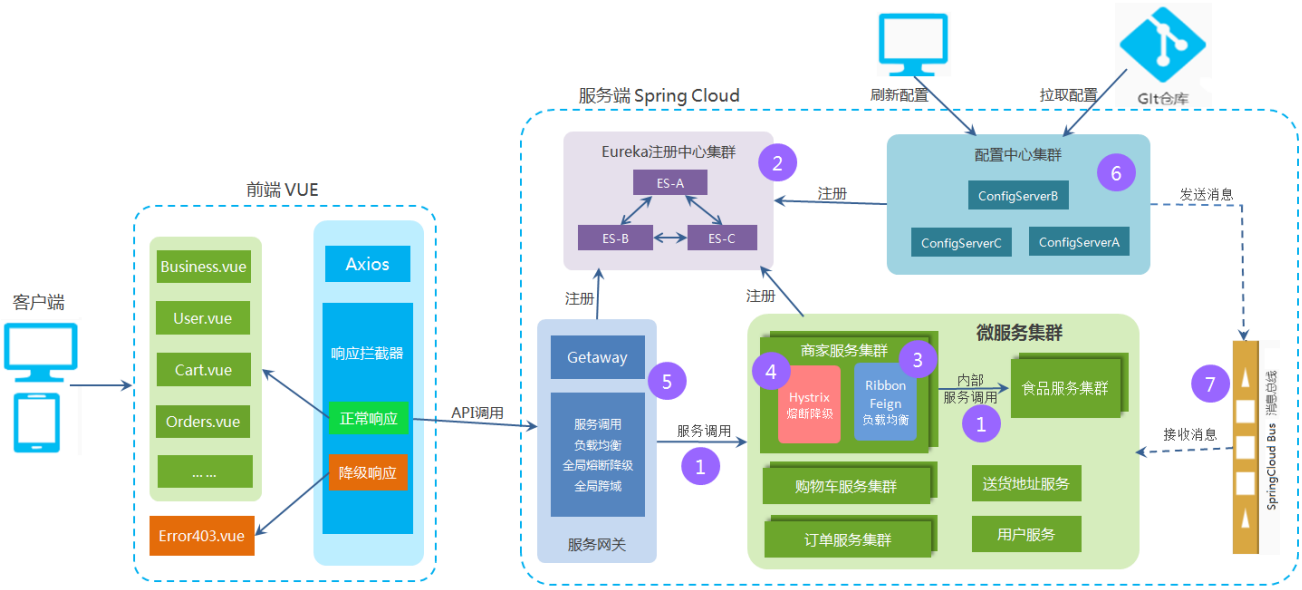
1.搭建Eureka高可用集群

本阶段项目完成总架构图中的第二阶段，添加Eureka高可用集群。



1.1.创建Eureka集群

在父工程下，创建 Maven Module 子工程（工程名：eureka_server_13000；Packaging：jar）

修改pom.xml文件

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.neusoft</groupId>
        <artifactId>spring_cloud_demo</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </parent>
    <artifactId>eureka_server_13000</artifactId>

    <dependencies>

        <!-- 添加 eureka server 依赖 -->
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
        </dependency>

        <!--热部署 gav -->
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

    </dependencies>
</project>
```

> 只需要添加netflix的eureka-server依赖即可创建一个Eureka注册中心

创建主启动类

```
package com.neusoft;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@EnableEurekaServer  //激活Eureka Server
public class MyApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

创建application.yml

```
server:
    port: 13000

#eureka配置
eureka:
    instance:
        hostname: eurekaServer13000   #集群版写法
    client:
        register-with-eureka: false   #是否将自己注册到eureka 服务当中（默认true）.
        fetch-registry: false         #是否启用从注册中心拉取服务列表的功能（默认true）.
        service-url:
            #这是集群版写法，13000注册给13001；反之，13001要注册给13000.
            defaultZone: http://eurekaServer13001:13001/eureka/
    server:
        enable-self-preservation: false     #关闭自我保护机制
```

在父工程下，创建 Maven Module 子工程（工程名：eureka_server_13001；Packaging：jar）。此工程与eureka_server_13000工程的内容一致，除了端口号之外。所以，eureka_server_13001工程的application.yml文件如下：

```yaml
server:
    port: 13001

#eureka配置
eureka:
    instance:
        hostname: eurekaServer13001   #集群版写法
    client:
        register-with-eureka: false   #是否将自己注册到eureka 服务当中（默认true）.
        fetch-registry: false         #是否启用从注册中心拉取服务列表的功能（默认true）.
        service-url:
            #这是集群版写法，13000注册给13001；反之，13001要注册给13000.
            defaultZone: http://eurekaServer13000:13000/eureka/
    server:
        enable-self-preservation: false     #关闭自我保护机制
```

1.2.修改hosts文件

打开本机的 C:\Windows\System32\drivers\etc\hosts 文件，添加映射信息

```hosts
## springcloud 配置
127.0.0.1 eurekaServer13000
127.0.0.1 eurekaServer13001
```

测试：启动Eureka集群服务，在浏览器地址栏中输入：http://localhost:13000/ 和 http://localhost:13001/，检查Eureka服务是否已经互相注册成功。

1.3.微服务注册到Eureka

给每个微服务的pom.xml文件，添加Eureka client 依赖：

```pom.xml
<!--加入eureka clinet的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

> 注意： 从SpringCloud的Edgware版本开始，工程中只要添加了此依赖，就成为Eureka Client工程。已经不需要在主启动类上配置@EnableEurekaClient或@EnableDiscoveryClient 注解了。

修改每个微服务的application.yml文件，添加Eureka配置：

```yaml
#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
    instance:
        prefer-ip-address: true        #使用ip地址向Eureka注册
        instance-id: ${spring.cloud.client.ip-address}:${server.port} #自定义微服务实例ID
```

测试：启动微服务，在Eureka Server管理后台中查看：所有微服务是否已经向Eureka注册成功。

1.4.获取服务列表并调用

商家微服务（服务消费者）可以通过Eureka Server获取元数据，通过获取的元数据就可以调用食品微服务（服务提供者）了。

修改business_server_10300工程的Controller类，使用DiscoveryClient接口获取元数据，并调用服务提供者。

```java
package com.neusoft.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
import org.springframework.cloud.client.discovery.DiscoveryClient;
import org.springframework.cloud.client.ServiceInstance;

import com.neusoft.po.Business;
import com.neusoft.po.CommonResult;
import com.neusoft.po.Food;
import com.neusoft.service.BusinessService;

@CrossOrigin("*")
@RestController
@RequestMapping("/BusinessController")
public class BusinessController {

    @Autowired
    private BusinessService businessService;

    @Autowired
```

```
    private RestTemplate restTemplate;

    @Autowired
    private DiscoveryClient discoveryClient;

    @GetMapping("/listBusinessByOrderTypeId/{orderTypeId}")
    public CommonResult<List> listBusinessByOrderTypeId(@PathVariable("orderTypeId") Integer
orderTypeId) throws Exception{
        List<Business> list = businessService.listBusinessByOrderTypeId(orderTypeId);
        return new CommonResult(200,"success (10200) ",list);
    }

    @RequestMapping("/getBusinessById/{businessId}")
    public CommonResult<Business> getBusinessById(@PathVariable("businessId") Integer
businessId) throws Exception{
        //通过服务提供者名（food-server）获取Eureka Server上的元数据
        List<ServiceInstance> instanceList =  discoveryClient.getInstances("food-server");
        //现在，元数据集合中只有一个服务信息（food-server）
        ServiceInstance instance = instanceList.get(0);

        Business business = businessService.getBusinessById(businessId);

        //使用DiscoveryClient获取元数据，主机地址与端口就可以不硬编码了
        CommonResult<List> result =
restTemplate.getForObject("http://"+instance.getHost()+":"+instance.getPort()+"/FoodController/l
istFoodByBusinessId/"+businessId, CommonResult.class);

        if(result.getCode()==200) {
            business.setFoodList(result.getResult());
        }
        return new CommonResult(200,"success (10200) ",business);
    }
}
```